# Go 语言笔试面试题(基础语法)

2020-09-04 22:10:10

*golang interview questions*

**Go 语言笔试面试题汇总， Github**

## Q1 = 和 := 的区别？

答案

`:=` 声明+赋值

`=` 仅赋值

```
var foo int
foo = 10
// 等价于
foo := 10
```

## Q2 指针的作用？

答案

指针用来保存变量的地址。

例如

```go
var x =  5
var p *int = &x
fmt.Printf("x = %d",  *p) // x 可以用 *p 访问
```

- * 运算符，也称为解引用运算符，用于访问地址中的值。
- &运算符，也称为地址运算符，用于返回变量的地址。

## Q3 Go 允许多个返回值吗？

答案

允许

```go
func swap(x, y string) (string, string) {
   return y, x
}

func main() {
   a, b := swap("A", "B")
   fmt.Println(a, b) // B A
}
```

## Q4 Go 有异常类型吗？

答案

Go 没有异常类型，只有错误类型（Error），通常使用返回值来表示异常状态。

```go
f, err := os.Open("test.txt")
if err != nil {
    log.Fatal(err)
}
```

## Q5 什么是协程（Goroutine）

答案

Goroutine 是与其他函数或方法同时运行的函数或方法。 Goroutines 可以被认为是轻量级的线程。 与线程相比，创建 Goroutine 的开销很小。 Go 应用程序同时运行数千个 Goroutine 是非常常见的做法。

## Q6 如何高效地拼接字符串

答案

Go 语言中，字符串是只读的，也就意味着每次修改操作都会创建一个新的字符串。如果需要拼接多次，应使用 `strings.Builder`，最小化内存拷贝次数。

```go
var str strings.Builder
for i := 0; i < 1000; i++ {
    str.WriteString("a")
```

```
}
fmt.Println(str.String())
```

## Q7 什么是 rune 类型

答案

ASCII 码只需要 7 bit 就可以完整地表示，但只能表示英文字母在内的 128 个字符，为了表示世界上大部分的文字系统，发明了 Unicode， 它是 ASCII 的超集，包含世界上书写系统中存在的所有字符，并为每个代码分配一个标准编号（称为 Unicode CodePoint），在 Go 语言中称之为 rune，是 int32 类型的别名。

Go 语言中，字符串的底层表示是 byte (8 bit) 序列，而非 rune (32 bit) 序列。例如下面的例子中 语 和 言 使用 UTF-8 编码后各占 3 个 byte，因此 `len("Go 语言")` 等于 8，当然我们也可以将字符串转换为 rune 序列。

```
fmt.Println(len("Go 语言")) // 8
fmt.Println(len([]rune("Go 语言"))) // 4
```

## Q8 如何判断 map 中是否包含某个 key？

答案

```
if val, ok := dict["foo"]; ok {
    //do something here
}
```

`dict["foo"]` 有 2 个返回值，val 和 ok，如果 ok 等于 `true`，则说明 dict 包含 key `"foo"`，val 将被赋予 `"foo"` 对应的值。

## Q9 Go 支持默认参数或可选参数吗？

答案

Go 语言不支持可选参数（python 支持），也不支持方法重载（java 支持）。

## Q10 defer 的执行顺序

答案

- 多个 defer 语句，遵从后进先出(Last In First Out，LIFO)的原则，最后声明的 defer 语句，最先得到执行。
- defer 在 return 语句之后执行，但在函数退出之前，defer 可以修改返回值。

例如：

```
func test() int {
    i := 0
    defer func() {
        fmt.Println("defer1")
    }()
```

```go
    defer func() {
        i += 1
        fmt.Println("defer2")
    }()
    return i
}

func main() {
    fmt.Println("return", test())
}
// defer2
// defer1
// return 0
```

这个例子中，可以看到 defer 的执行顺序：后进先出。但是返回值并没有被修改，这是由于 Go 的返回机制决定的，执行 return 语句后，Go 会创建一个临时变量保存返回值，因此，defer 语句修改了局部变量 i，并没有修改返回值。那如果是有名的返回值呢？

```go
func test() (i int) {
    i = 0
    defer func() {
        i += 1
        fmt.Println("defer2")
    }()
    return i
}

func main() {
    fmt.Println("return", test())
}
// defer2
// return 1
```

这个例子中，返回值被修改了。对于有名返回值的函数，执行 return 语句时，并不会再创建临时变量保存，因此，defer 语句修改了 i，即对返回值产生了影响。

## Q11 如何交换 2 个变量的值？

答案

```go
a, b := "A", "B"
a, b = b, a
fmt.Println(a, b) // B A
```

## Q12 Go 语言 tag 的用处？

答案

tag 可以理解为 struct 字段的注解，可以用来定义字段的一个或多个属性。框架/工具可以通过反射获取到某个字段定义的属性，采取相应的处理方式。tag 丰富了代码的语义，增强了灵活性。

例如：

```go
package main

import "fmt"
import "encoding/json"

type Stu struct {
    Name string `json:"stu_name"`
    ID   string `json:"stu_id"`
    Age  int    `json:"-"`
}

func main() {
    buf, _ := json.Marshal(Stu{"Tom", "t001", 18})
    fmt.Printf("%s\n", buf)
}
```

这个例子使用 tag 定义了结构体字段与 json 字段的转换关系，Name -> `stu_name`, ID -> `stu_id`，忽略 Age 字段。很方便地实现了 Go 结构体与不同规范的 json 文本之间的转换。

## Q13 如何判断 2 个字符串切片（slice) 是相等的？

答案

go 语言中可以使用反射 `reflect.DeepEqual(a, b)` 判断 a、b 两个切片是否相等，但是通常不推荐这么做，使用反射非常影响性能。

通常采用的方式如下，遍历比较切片中的每一个元素（注意处理越界的情况）。

```go
func StringSliceEqualBCE(a, b []string) bool {
    if len(a) != len(b) {
        return false
    }

    if (a == nil) != (b == nil) {
        return false
    }

    b = b[:len(a)]
    for i, v := range a {
        if v != b[i] {
            return false
        }
```

```
    }

    return true
}
```

## Q14 字符串打印时，%v 和 %+v 的区别

答案

%v 和 %+v 都可以用来打印 struct 的值，区别在于 %v 仅打印各个字段的值，%+v 还会打印各个字段的名称。

```
type Stu struct {
    Name string
}

func main() {
    fmt.Printf("%v\n", Stu{"Tom"}) // {Tom}
    fmt.Printf("%+v\n", Stu{"Tom"}) // {Name:Tom}
}
```

但如果结构体定义了 `String()` 方法，%v 和 %+v 都会调用 `String()` 覆盖默认值。

## Q15 Go 语言中如何表示枚举值(enums)

答案

通常使用常量(const) 来表示枚举值。

```
type StuType int32

const (
    Type1 StuType = iota
    Type2
    Type3
    Type4
)

func main() {
    fmt.Println(Type1, Type2, Type3, Type4) // 0, 1, 2, 3
}
```

参考 What is an idiomatic way of representing enums in Go? - StackOverflow

## Q16 空 struct{} 的用途

答案

使用空结构体 struct{} 可以节省内存，一般作为占位符使用，表明这里并不需要一个值。

```go
fmt.Println(unsafe.Sizeof(struct{}{})) // 0
```

比如使用 map 表示集合时，只关注 key，value 可以使用 struct{} 作为占位符。如果使用其他类型作为占位符，例如 int，bool，不仅浪费了内存，而且容易引起歧义。

```go
type Set map[string]struct{}

func main() {
    set := make(Set)

    for _, item := range []string{"A", "A", "B", "C"} {
        set[item] = struct{}{}
    }
    fmt.Println(len(set)) // 3
    if _, ok := set["A"]; ok {
        fmt.Println("A exists") // A exists
    }
}
```

再比如，使用信道(channel)控制并发时，我们只是需要一个信号，但并不需要传递值，这个时候，也可以使用 struct{} 代替。

```go
func main() {
    ch := make(chan struct{}, 1)
    go func() {
        <-ch
        // do something
    }()
    ch <- struct{}{}
    // ...
}
```

再比如，声明只包含方法的结构体。

```go
type Lamp struct{}

func (l Lamp) On() {
        println("On")

}
func (l Lamp) Off() {
        println("Off")
}
```